Practice : Bellman-Ford

Deadline: May.27/28/29. Must be checked onsite (during labs).Online checks via email or messaging apps are not accepted.

---

1. Objective

This Practice has one question based on **Chapter 6.8 Shortest Paths** in the lecture. You need understand the application of the Bellman-Ford algorithm in finding shortest paths, and optimize its time and space complexity as much as possible.

2. Task

bellman-Ford algorithm solves the **single-source shortest path problem** on a weighted graph with **negative edge weights**.

Description:

Given n nodes, m edges, and the weight of each edge (weights can be negative integers), find the shortest path from node 0 to every other node.

Input Description:

The first line contains two integers **n** and **m**, representing n nodes and m edges, separated by a space.

The next **m** lines each contain three integers: the first two integers represent the node indices, and the third integer represents the **weight** between the two nodes.

Output Description:

If there is a negative cycle, output -1.

Else output two lines:

First line: Output the shortest distances from node 0 to the other nodes in order, separated by a space. If there is no path, output 2,147,483,647. (The max value of Integer)

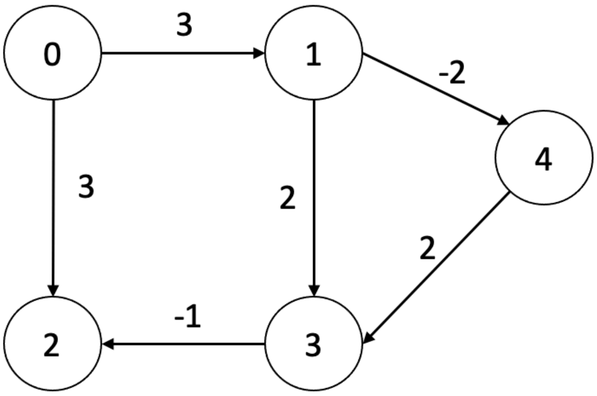
Second line: Output the father node indice of current node in the shortest path. If there is no path can visit the node or the index of the node is 0, output -1.

Input Test case1:

```
5 6
1 4 -2
1 3 2
0 1 3
3 2 -1
4 3 2
0 2 3
```

Output1

```
0 3 2 3 1
-1 0 3 4 1
```

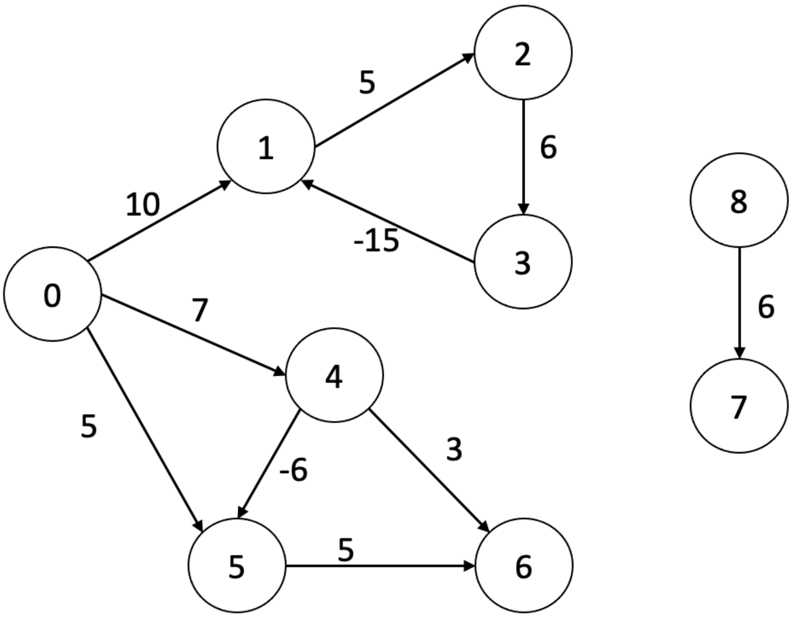


Input Test case2:

```
9 10
0 1 10
1 2 5
2 3 6
0 4 7
0 5 5
3 1 -15
4 5 -6
4 6 3
5 6 5
8 7 6
```

Output2

```
-1
```



Output Explanation:

First line: Nodes 1, 2, and 3 form a negative cycle, so output -1.

Input Test case3:

In the file `1.in`

Output Test case3:

Dist:

```
0 3 4 7 9 1 2147483647 13 9 8 14 10 -3 12 33 3 24 26 11 7 12 31 19 6 4 19 7 22
8 29 21 -4 2 6 5 13 1 15 13 2 5 11 11 9 5 3 8 2 17 12 9 11 9 9 14 12 9 14 15 7
5 3 8 11 14 8 6 8 4 10 10 11 7 12 7 -7 12 8 11 23 11 6 23 10 7 14 8 8 14 10 10
11 -4 7 12 10 2 ......
```

Parent:

```
-1 0 0 0 0 0 -1 19 2 0 8 2 2 11 0 5 3 1 9 12 8 4 13 5 12 5 0 2 12 4 8 1 0 2 12
0 12 0 9 13 12 8 1 39 32 36 31 12 43 18 12 9 9 36 5 53 31 50 11 24 15 5 23 53
47 31 44 36 32 32 40 59 23 34 40 ......
```

3. Introduction

3.1. Key Data Structure: `dist[]` Array

- **Definition**: `dist[i]` = the **shortest distance** from the **source node (0)** to node `i`
- Initialization
  - `dist[0] = 0` (distance to source itself is 0)

- `dist[i] = INT_MAX (2147483647)` for all `i ≠ 0` (unreachable initially)

3.2. Dynamic Transition Equation

Traverse all edges for n-1 time, and in each time:

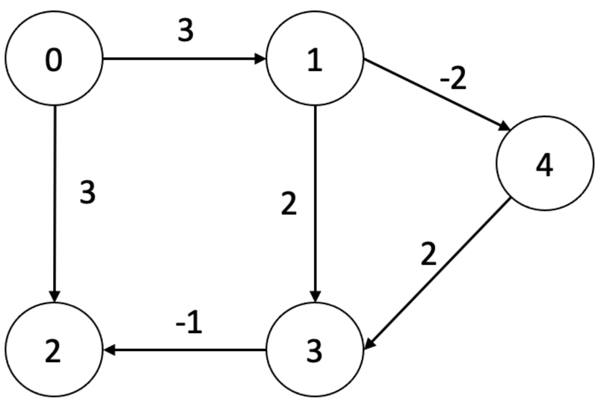
For **every edge `u → v` with weight `w`**:

```
if dist[u] is reachable AND dist[v] > dist[u] + w
    dist[v] = dist[u] + w
    parent[v] = u  (update the predecessor node)
```

- **Logic**: We can get a shorter path to `v` by going through `u`.

Using test case 1 as an example:

```
5 6
1 4 -2
1 3 2
0 1 3
3 2 -1
4 3 2
0 2 3
```



time\dist	0	1	2	3	4
Original	0	2147483647	2147483647	2147483647	2147483647
1	0	3	3	2147483647	2147483647
2	0	3	3	3	1
3	0	3	2	3	1
4	0	3	2	3	1

3.3. Negative Cycle Detection Rule

Relax all edges again, if find one value of dist[v] is smaller, there would be a negative cycle.

3.4. Predecessor (`parent[]`) Array

- `parent[i]` = the previous node on the shortest path from `0` to `i`
- Initialization: `parent[i] = -1` for all nodes
- Update only when `dist[v]` is refreshed by the transition equation

4. Submission

Show the following to your tutor or SA **onsite**:

Grading:

- Submissions on time:
  - 0.5 points for finding correct weight of two test cases.
  - 0.3 points for finding final selected jobs of two test case 3.
  - 0.2 point for answer questions like:
    - Can you give an example to explain why Dijkstra's algorithm is not feasible in the presence of negative edges?
    - How do you detect whether a negative cycle exists? If you want to find all nodes that are part of any negative cycle, how would you improve your current code?
- Late submissions: 0 points.